

Service Analysis: Create Outward Transfer

T24 R20/FT → TPH R25/IRIS API Migration Audit

Source Routine: CIB.OT.FT

Type: OFS Wrapper Routine (Custom Call Routine)

Current Version: T24 R20 / FT (OFS)

Target: TPH R25 / IRIS API

Used By (per code): Channels via CHNL.CODE — includes API, VOS, BPM, BB.MCP, and generic channels (channel-agnostic router)

Service ID (SRVC.ID): 1006 — parsed from header segment 1 (or segment 1 of the '#'-delimited sub-header when HEADER.ID starts with 'XML') and stored against every CIB.API.FT record for this service

Document Control

Field	Value
Document Title	Service Analysis — Create Outward Transfer (CIB.OT.FT)
Version	1.1
Status	Draft — pending peer review
Prepared For	T24 R20 → R25 / FT → TPH Migration Programme
Prepared Basis	Static code review of CIB.OT.FT (as supplied)
Last Updated	23-Aug-2026
Related Documents	CIB.API.VALIDATE.FT audit (pending), CIB.API.OVERRIDE table export (pending), ST.CIB.FI.FT.PARAM table export (pending)

Executive Summary

CIB.OT.FT is a high-complexity, bespoke OFS orchestration routine with no direct 1:1 equivalent on TPH/IRIS. It combines input parsing, multi-layer business validation, config-table-driven rule branching (per channel and per customer industry), currency conversion, an idempotency cache, and post-posting auto-authorization. Migration risk is driven less by the OFS-to-API field mapping itself and more by the amount of business logic and control (limits, overdraft, restricted-industry posting blocks, auto-auth) currently embedded in Universe/BASIC code and config tables rather than in T24's native validation layer.

Risk & Priority Rating

Dimension	Rating	Rationale
Business Criticality	High	Handles live outward funds transfer posting — a core financial (money-movement) transaction
Migration Complexity	High	51 input fields, 4 custom status codes, 8+ external routine dependencies, 4 config tables, channel-specific branching
Control/Compliance Sensitivity	High	Contains overdraft authorization bypass, auto-authorization, and industry-based posting restriction logic — all control-relevant

Dimension	Rating	Rationale
Testing Priority	P1 — test first	Financial transaction with multiple silent business-rule branches; regression coverage must include every channel (API, VOS, BPM, BB.MCP) and every status code path

Service Description (Business View)

"Create Outward Transfer" lets a customer or bank operator send money out of a CIB account to a beneficiary — whether that beneficiary holds an account at CIB, at another bank locally, or at a bank abroad. It covers the everyday "send money" journey behind channels such as internet/mobile banking, the corporate/business banking portal (BB.MCP), branch/teller-assisted transfers (BPM), and other API-driven integrations (VOS and generic API clients).

In plain terms, when this service is called it:

- Confirms the request hasn't already been processed (protects against double-submission, e.g. a channel retry or network timeout causing the same payment to be sent twice).
- Checks whether the requesting channel or transaction type is currently allowed to transact (e.g. temporarily blocked channels, restricted transaction types).
- For certain business/corporate customers, checks whether their industry or account is subject to a posting restriction (e.g. a customer flagged for enhanced control who cannot self-initiate certain payments) and blocks the transfer with an explanation if so.
- Confirms the customer has enough available funds to cover the transfer — converting currency if the account and the transfer are in different currencies — and, depending on the channel's configuration, may allow the transfer to proceed into an overdraft with a hold for review rather than rejecting it outright.
- Validates the beneficiary's account/IBAN and correctly identifies the beneficiary's bank, intermediary bank, and any routing details needed to route the payment domestically or internationally (including SWIFT-style MT/MX name formatting for cross-border payments).
- Works out who pays the transfer charges (the sender, the beneficiary, or shared) and applies the correct commission code and amount.
- Submits the payment for posting in the core banking ledger and, where the channel is configured to allow it, automatically authorizes the transaction so the customer doesn't need to wait for a second banker to approve it.
- Sends back a clear outcome to the channel: success with a reference number, or a rejection with a specific reason (insufficient funds, restricted, blocked, etc.) that the channel can show to the customer.

Who uses it: retail and corporate customers initiating transfers via digital channels, and CIB staff/branch systems initiating transfers on a customer's behalf. Typical triggers: "Send Money" / "Transfer Funds" / "International Wire" actions in the mobile app, internet banking, the Business Banking portal, or a partner system integrating via API.

1. Purpose

Takes a pipe-delimited (|) flat string (I.PARAM) built by the channel, parses it into ~45 fields, builds an OFS record, and posts it to FUNDS.TRANSFER via OFS.BULK.MANAGER. Returns a pipe-delimited response (O.MSG) with status code, FT ID, raw OFS record, paid amount, and overrides.

This is not a T24 standard API — it's a bespoke OFS orchestration wrapper. This is exactly the kind of routine that has no direct 1:1 equivalent in IRIS APIs, so it needs careful field-by-field and rule-by-rule mapping.

2. Input Contract (I.PARAM — positional, pipe-delimited)

Not every field taken from the input is written into the OFS record posted to FUNDS.TRANSFER. The "Posted to OFS?" column below shows exactly what happens to each field: some are posted as ledger data, some only flow into the OFS service-call header, some are used purely for internal validation/control-flow and never posted at all, and a few show no downstream reference in this routine and need verification before being assumed safe to drop.

#	Field	Posted to OFS?	Notes
Header	SRVC.ID#CHNL.CODE#MSG.ID#TOKEN.ID	Posted (OFS header/routing only)	Used to key CIB.API.FT audit record and drive channel-config lookups; not itself an OFS data field
1	TXN.TYPE	NOT posted — control-flow/validation only	Used only as input to CIB.API.VALIDATE.FT restriction check — never written to the OFS record
2	BOOK.DATE	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify: dead field, or used in logic not shown here
3	VALUE.DATE	Posted to OFS (data field)	→ CREDIT.VALUE.DATE (defaults to TODAY if blank)
4	BRANCH.CODE	Posted (OFS header/routing only)	Used in the OFS header/routing segment of OFSRECORD, not as a named FT field
5	PROFIT.CNTR	NOT posted — unreferenced downstream (verify)	Split into PROFIT.CNTR.CUST/PROFIT.CNTR.DEBT but neither variable is referenced again downstream — verify
6	SRC.ACC.ID	Posted to OFS (data field)	→ DEBIT.ACCT.NO; also drives available-balance calculation
7	SRC.CURR	Posted to OFS (data field)	→ CREDIT.CURRENCY; also used for cross-currency check
8	SRC.AMT	Posted to OFS (data field)	→ CREDIT.AMOUNT; also used in the available-balance comparison
9	SRC.NARR	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify
10	BEN.NAME	Posted to OFS (data field)	→ BEN.CUSTOMER (multi-part, MT → MX transliteration for non-EGP)
11	BEN.ACC	Posted to OFS (data field)	→ IBAN.BEN or BEN.ACCT.NO depending on IBAN validation result
12–14	BEN.ADDR.1/2/3	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify
15	CHARGE.IND	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify
16	BEN.BK.NAME	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify (only BEN.BIC.CD is actually posted)
17	BEN.BIC.CD	Posted to OFS (data field)	→ BNK.NME.57

#	Field	Posted to OFS?	Notes
		field)	
18	BEN.BK.SORT.C D	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify
19	BEN.BK.BRANC H	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify
20	BEN.BK.BRN.CD	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify
21	BEN.NARRATIV E	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify (distinct from FT.NARRATIVE, which IS posted)
22	BEN.OUR.CHAR GES	Posted to OFS (data field)	Posted as-is; also drives COMMISSION.CODE derivation logic
23	COMMISSION.C ODE	Posted to OFS (data field)	Posted, but may be overwritten/cleared by the OUR/BEN/SHA + account-length rule
24	COMMISSION.T YPE	Posted to OFS (data field)	→ COMMISSION.TYPE:n (looped, multivalue)
25	COMMISSION.A MOUNT	Posted to OFS (data field)	→ COMMISSION.AMT:n (looped, paired with above)
26	GTS.CONTROL	Posted (OFS header/routing only)	Used in the OFS header/routing segment of OFSRECORD, not as a named FT field
27	AUTH.FLAG	Posted (OFS header/routing only)	Used in the OFS header/routing segment; also drives control-flow (overwritten to 1 if AUTH.FLAG.NEW set)
28	OD.TYPE	NOT posted — control- flow/validation only	Control-flow only — gates the balance check (AVAL.BALANCE >= SRC.AMT OR OD.TYPE='Y'); never written to OFS
29	ORDERING.CUS T	Posted to OFS (data field)	→ ORDERING.CUST:n (looped) and ORD.CUST.ACCT
30	ORDERING.BAN K	Posted AND repurposed for control-flow (dual- use)	→ ORDERING.BANK:n (posted); also silently repurposed as the AUTH.FLAG.NEW source when no channel override routine fires — a dual-use field
31	CREDIT.ACCT.N O	Posted to OFS (data field)	Posted as-is
32	CUSTOMER.SPR EAD	NOT posted — unreferenced downstream (verify)	Parsed but no further reference found downstream — verify
33	CUSTOMER.RAT E	Posted to OFS (data field)	Posted; zeroed out if channel = BPM and value = 0
34	FREE.MARKT.C ODE	Posted to OFS (data field)	Posted as-is

#	Field	Posted to OFS?	Notes
35	FT.NARRATIVE	Posted to OFS (data field)	→ DEBIT.THEIR.REF
36	INTERMED.BANK.ACCT	Posted to OFS (data field)	→ INTERMED.BANK.ACC
37	INTERMED.BANK	Posted to OFS (data field)	→ INTERMED.BANK:n (looped)
38	ACCT.WITH.BANK.ACC	Posted to OFS (data field)	Posted as-is
39	ACCT.WITH.BANK	Posted to OFS (data field)	→ ACCT.WITH.BANK:n (looped, capped at 4; SC/BL/CH/FW prefix escaping)
40	PAYMENT.DETAILS	Posted to OFS (data field)	→ PAYMENT.DETAILS:n (looped, capped at 4)
41	BK.TO.BK.OUT	Posted to OFS (data field)	Posted via FILL.SUB.VALUE merge logic against the existing FT record
42	FUNCTION.ID	Posted (OFS header/routing only)	Used in the OFS header/routing segment (VALIDATE vs PROCESS mode) — not a named FT field
43	FT.TXN.NARR	Posted to OFS (data field)	→ FT.TXN.NARR::
44	CHRG.ACCT.NO	Posted to OFS (data field)	→ CHARGES.ACCT.NO
45	CREDIT.THEIR.REF	Posted to OFS (data field)	Posted as-is
46	DEBIT_VALUE.DATE	Posted to OFS (data field)	→ DEBIT.VALUE.DATE (defaults to VALUE.DATE if blank)
47	VERSION.NAME	Posted (OFS header/routing only)	Used in the OFS header/routing segment (selects the FT version template) — not a named FT field
48	OLD.CBE.CODE	Posted to OFS (data field)	Posted as-is (CBE regulatory reporting field)
49	NEW.CBE.CODE	Posted to OFS (data field)	Posted as-is (CBE regulatory reporting field)
50	SELL.CODE	Posted to OFS (data field)	Posted as-is (CBE regulatory reporting field)
51	CBE.NAR.AR	Posted to OFS (data field)	Posted as-is (CBE regulatory reporting field, Arabic narrative)

Pre-processing: input is sanitized — &, /, ', ' , " , (,) , { , } , : are all replaced with safe placeholder chars (- or ^) before parsing. This is a hidden validation/normalization layer that must be replicated in TPH/IRIS if the same special-character handling is expected.

2.1 Fields Received But NOT Posted to the Core Ledger — Needs a Decision

The following input fields are accepted from the channel but, based on this routine's code, never make it into the FUNDS.TRANSFER posting. For each, the migration team must explicitly decide: (a) intentionally drop it in the

new IRIS request contract, (b) discover where else it's used in T24 R20 (a routine not covered by this file) before dropping it, or (c) it was already dead code that predates the current requirements and can be safely retired.

- BOOK.DATE — parsed, no downstream reference found
- PROFIT.CNTR — split into CUST/DEBT components, neither referenced again
- SRC.NARR — parsed, no downstream reference found
- BEN.ADDR.1, BEN.ADDR.2, BEN.ADDR.3 — beneficiary address lines parsed, never posted or used
- CHARGE.IND — parsed, no downstream reference found
- BEN.BK.NAME — parsed; only the BIC code (BEN.BIC.CD) actually gets posted, the bank name text does not
- BEN.BK.SORT.CD, BEN.BK.BRANCH, BEN.BK.BRN.CD — beneficiary bank routing detail fields parsed, never posted or used
- BEN.NARRATIVE — parsed but distinct from FT.NARRATIVE (which is posted) — this specific field is dropped
- CUSTOMER.SPREAD — parsed, no downstream reference found

Separately, two fields are consumed only for validation/control-flow and are correctly never posted as ledger data — this is expected behavior, not a gap: TXN.TYPE (restriction check input) and OD.TYPE (overdraft balance-check gate).

2A. Technical Processing Flow (Sequence)

Step-by-step execution order as implemented in CIB.OT.FT, useful for building an equivalent orchestration sequence (e.g. BPEL/API-gateway flow) in TPH:

1. OPEN.FILES — opens all required file handles (F.ACCOUNT.RETAIL.CU.ACCT, F.COMPANY, FBANK.NOSTRO.ACC, F.ACCOUNT, F.CURRENCY, FBANK.FUNDS.TRANSFER, FBANK.FUNDS.TRANSFER\$NAU, FBANK.CIB.API.FT, F.CIB.ONLINE.FTS, F.CIB.API.OVERRIDE, F.ST.CIB.FI.FT.PARAM, FBANK.CUSTOMER).
2. Sanitize I.PARAM — strip/replace reserved delimiter characters so downstream FIELD() parsing is safe.
3. SAVE.MSG — parse header (SRVC.ID, CHNL.CODE, MSG.ID, TOKEN.ID), resolve per-channel override config (CIB.API.OVERRIDE) and Business Banking industry flag (CHECK.FI.IND.UPD.FLAG / ST.CIB.FI.FT.PARAM), parse all 51 positional fields, then read-or-create the CIB.API.FT audit/idempotency record for this MSG.ID.
4. Idempotency gate — if CIB.API.FT already shows CUR.STATUS = 0000 for this MSG.ID, skip straight to building the cached response (no re-validation, no re-posting).
5. Channel restriction gate (API channel only) — call CIB.API.VALIDATE.FT; if restricted, short-circuit with status 0006.
6. FT.PROCESS — normalize SRC.ACC.ID format, compute available balance (GET.NET.AVL.BL), convert currency if needed (GET.EQ.AMOUNT / EXCHRATE), apply overdraft/auth-flag logic.
7. Business Banking gate — if BB.FLAG is set and a posting-restriction match is found, short-circuit with status 3003 before the balance check is even evaluated.
8. Balance gate — if available balance is insufficient and no overdraft override applies, short-circuit with status 0011 (Business Banking) or 0006 (standard).
9. Build OFSRECORD — assemble the full OFS string field-by-field (see 2B below), including BIC/IBAN resolution, commission-code derivation, and CBE reporting fields.
10. BB.MCP dry-run gate — for BB.MCP channel only, run a VALIDATE-mode OFS call first; if ACCT.UNAUTH.OD override is returned, short-circuit with status 0011.
11. Post — call OFS.INITIALISE.SOURCE + OFS.BULK.MANAGER to submit the real (PROCESS-mode) OFS record to FUNDS.TRANSFER.

12. Parse OFS response — extract success flag, FT.ID, and status; on success set PAID.AMOUNT and call UPDATE.FTS.ONLINE (SMS trigger + journal).
13. Re-read FUNDS.TRANSFER (NAU or live) to fetch OVERRIDE text for the response payload.
14. Conditional auto-authorization — if channel config requires it and the record is in an auth-pending state, call CIB.MSG.AUTHORIZE.SC.
15. UPDATE.MSG — persist final CUR.STATUS, FT.ID, OFSRECORD, PAID.AMOUNT back to CIB.API.FT (skipped when FUNCTION.ID = VALIDATE, i.e. dry-run/enquiry calls don't overwrite the audit record).
16. Return O.MSG to caller in the format HEADER.ID|STATUS|FT.ID|OFSRECORD|PAID.AMOUNT|OVERRIDE|.

2B. OFS Record Field Construction Map

Shows exactly how each field posted to FUNDS.TRANSFER (via OFS) is derived. This is the field-by-field spec needed to build the equivalent IRIS API request payload mapping.

OFS Field	Source Field(s)	Transform / Rule
DEBIT.ACCT.NO	SRC.ACC.ID	Normalized (short-format account re-mapped via F.ACCOUNT.RETAIL.CU.ACCT lookup if ≤10 chars)
CREDIT.ACCT.NO	CREDIT.ACCT.NO	Passed through as-is
CREDIT.CURRENCY / CREDIT.AMOUNT	SRC.CURR / SRC.AMT	Direct pass-through
CREDIT.VALUE.DATE	VALUE.DATE	Defaults to TODAY if blank
DEBIT.VALUE.DATE	DEBIT_VALUE.DATE	Defaults to VALUE.DATE if blank
ORDERING.CUST:n	ORDERING.CUST	'_' split into multivalue; '~' → '_' unescape per line
ORD.CUST.ACCT	ORDERING.CUST (first value)	Only set when ORDERING.CUST is non-blank AND SRC.ACC.ID length > 12
PAYMENT.DETAILS:n	PAYMENT.DETAILS	'_' split, capped at 4 lines
BEN.CUSTOMER:n	BEN.NAME	Reformatted: first two name parts merged, MT → MX transliteration via CIB.MT.MX for non-EGP, then parts 3–4 re-appended; capped at 4 lines
IBAN.BEN or BEN.ACCT.NO	BEN.ACC	IBAN.BEN used if 29-char value passes IBANService.ValidateIBAN; otherwise BEN.ACCT.NO
ACCT.WITH.BANK:n	ACCT.WITH.BANK	'_' split, capped at 4; SC/BL/CH/FW-prefixed codes get '^' escape prefix
INTERMED.BANK:n	INTERMED.BANK	'_' split into multivalue
BNK.NME.57	BEN.BIC.CD	Direct mapping
INTERMED.BANK.ACC	INTERMED.BANK.ACCT	Direct pass-through
CUSTOMER.RATE	CUSTOMER.RATE	Zeroed if channel = BPM and incoming value = 0
ORDERING.BANK:n	ORDERING.BANK	'_' split; minimum 1 line enforced even if blank
DEBIT.THEIR.REF	FT.NARRATIVE	Direct mapping
CREDIT.THEIR.REF	CREDIT.THEIR.REF	Direct pass-through

OFS Field	Source Field(s)	Transform / Rule
ACCT.WITH.BANK.ACC	ACCT.WITH.BANK.ACC	Direct pass-through
FREE.MARKT.CODE	FREE.MARKT.CODE	Direct pass-through
BK.TO.BK.OUT:1:n	BK.TO.BK.OUT (merged with existing FT record via FILL.SUB.VALUE)	'NULL' literal substituted when incoming value blank; merged multivalued vs current record's BK.TO.BK.OUT
BEN.OUR.CHARGES	BEN.OUR.CHARGES	Direct pass-through
COMMISSION.CODE	Derived from BEN.OUR.CHARGES	OUR/SHA → DEBIT PLUS CHARGES, BEN → CREDIT LESS CHARGES; cleared blank if DEBIT PLUS CHARGES and SRC.ACC.ID length > 12
CHARGES.ACCT.NO	CHRG.ACCT.NO	Direct pass-through
FT.TXN.NARR::	FT.TXN.NARR	Direct pass-through (double-colon = no-index field)
COMMISSION.TYPE:n / COMMISSION.AMT:n	COMMISSION.TYPE / COMMISSION.AMOUNT	Looped together, '_' split multivalued
SRC.CHANNEL	CHNL.CODE (first 4 chars)	Truncated
INT.MSG.NUM.1	MSG.ID	Direct mapping
UETR.USR.INP	MSG.ID	Only set when CHNL.CODE = VOS
OLD.CBE.CODE / NEW.CBE.CODE / SELL.CODE	OLD.CBE.CODE / NEW.CBE.CODE / SELL.CODE	Direct pass-through (CBE regulatory reporting fields)
CBE.NAR.AR	CBE.NAR.AR	Direct pass-through (Arabic narrative for CBE reporting)

2C. Files Read / Written

File / Table	Access	Purpose
F.ACCOUNT.RETAIL.CU.ACCCT	Read	Resolve short-format retail account numbers to full account ID
F.COMPANY	Read	Opened but not directly referenced in the reviewed logic — confirm usage elsewhere/legacy
FBNK.NOSTRO.ACC	Read	Opened; nostro handling context (not directly read in this excerpt)
F.ACCOUNT	Read	Debit account currency lookup, customer number lookup for industry check
F.CURRENCY	Read	Opened for currency-context; conversion itself delegated to EXCHRATE
FBNK.FUNDS.TRANSFER	Read	Post-posting re-read to fetch OVERRIDE text; idempotency check pre-read
FBNK.FUNDS.TRANSFER\$N AU	Read	Un-authorized/in-flight FT record read (before authorization completes)

File / Table	Access	Purpose
FBNK.CIB.API.FT	Read/Write	Core idempotency & audit store — one record per MSG.ID, updated at start and end of processing
F.CIB.ONLINE.FTS	Read/Write	SMS/online-transfer notification log, written on successful post
F.CIB.API.OVERRIDE	Read	Per-channel configuration: auth-flag behavior, OD-flag behavior, restricted posting codes, extra validation routine name
F.ST.CIB.FI.FT.PARAM	Read	Per-channel list of customer industry codes subject to Business Banking restriction
FBNK.CUSTOMER	Read	Customer industry code and posting-restriction code lookup

3. Validations & Business Rules (the core of what must migrate)

3.1 Idempotency / Duplicate Message Check

- Every inbound MSG.ID is looked up in CIB.API.FT table.
- If a prior record exists with CUR.STATUS = '0000' (already successfully processed), the routine does not reprocess — it returns the cached FT ID / OFS record / paid amount from the original run.

Gap risk: IRIS APIs typically have their own idempotency-key handling — must confirm it matches this exact "return cached success, skip reprocessing" behavior, not just "reject duplicate."

3.2 Channel-Level Restriction Check (API channel only)

- If CHNL.CODE = 'API', calls external routine CIB.API.VALIDATE.FT with SRC.ACC.ID | CHNL.CODE | TXN.TYPE.
- Returns RESTRICT.FLAG / RESTRICT.DESC.
- If RESTRICT.FLAG = '1' → immediate reject with error 0006 + description, before any balance or posting logic runs.

This is a black-box external validation — its internal rules are not visible in this routine and must be independently audited (separate service/routine).

3.3 Customer Industry-Based Posting Restriction (Business Banking)

- Reads F.ST.CIB.FI.FT.PARAM keyed by first 4 chars of channel code.
- Gets the debit account's customer, reads CUSTOMER.INDUSTRY.
- If industry code matches a configured restricted-industry list → sets BB.FLAG = "TRUE", forces OD.TYPE = "N" (no overdraft allowed).
- If BB.FLAG = TRUE: checks customer/account POSTING.RESTRICT codes against a configured override list (CIB.API.OVERRIDE). If a match is found → hard reject with error 3003 and a configured description — transaction never reaches balance check or posting.

Gap risk: This entire industry+restriction-code rule engine is config-table-driven (ST.CIB.FI.FT.PARAM, CIB.API.OVERRIDE) — needs to be reproduced either in TPH parameter tables or in an orchestration/pre-validation layer in front of IRIS.

3.4 Available Balance Check

- Calls GET.NET.AVL.BL to compute limit balance, lock amount, book/clear/available balance for the debit account.

- If debit currency $\neq$ transaction currency, converts available balance to transaction currency via EXCHRATE (market code 11).
- Rule: proceed only if AVAL.BALANCE $\geq$ SRC.AMT OR OD.TYPE = 'Y' (overdraft allowed).
- Else $\rightarrow$ reject: 0011 if BB.FLAG = TRUE (Business Banking insufficient balance); 0006 otherwise (generic insufficient balance).
- Both return the computed available balance in the error description.

3.5 Overdraft Authorization Flag Logic

- If CHANGE.AUTH.FLAG = "YES" (from CIB.API.OVERRIDE config per channel) and AVAL.BALANCE < SRC.AMT: sets OD.FLAG = "Y", forces the record into an authorization-required state (AUTH.FLAG = "1", AUTH.FLAG.NEW = "1").

This means: certain channels are configured to allow overdraft but force a maker-checker style hold rather than an outright reject.

3.6 BB.MCP Overdraft Charge Dry-Run Check

- Specifically for channel BB.MCP: before real posting, does a VALIDATE-mode dry-run OFS call against the built OFS record.
- If the dry-run response contains override ACCT.UNAUTH.OD $\rightarrow$ hard reject with 0011 (insufficient balance), even if the earlier balance check passed. This is effectively a second, stricter balance/overdraft gate specific to this channel.

3.7 IBAN Validation

- If beneficiary account length = 29 $\rightarrow$ validated via IBANService.ValidateIBAN.
- If valid $\rightarrow$ sent as IBAN.BEN; if invalid or not IBAN-length $\rightarrow$ sent as plain BEN.ACCT.NO.

3.8 Charges/Commission Rule Logic

- BEN.OUR.CHARGES drives default COMMISSION.CODE when not explicitly supplied: OUR $\rightarrow$ DEBIT PLUS CHARGES; BEN $\rightarrow$ CREDIT LESS CHARGES; SHA $\rightarrow$ DEBIT PLUS CHARGES
- Extra override: if resulting code is DEBIT PLUS CHARGES AND SRC.ACC.ID length > 12 $\rightarrow$ commission code is cleared entirely (blank). This is a non-obvious business rule tied to account number format/type.

3.9 Auto-Authorization (Maker-Checker Bypass)

- After successful posting, if CHANGE.AUTH.FLAG = "YES" and AUTH.FLAG = 1 and AUTH.FLAG.NEW = "0": automatically calls CIB.MSG.AUTHORIZE.SC to auto-authorize the transaction (system-driven authorization, not a human checker).

This is a critical control to validate in TPH — confirm whether auto-auth is still config-driven per channel or hardcoded differently.

3.10 SMS / Online FT Journal Update

- On success, checks CIB.CHECK.SMS.RUN to decide whether to log the FT into CIB.ONLINE.FTS (used presumably for SMS notification triggers) and fires JOURNAL.UPDATE.

4. Response Contract (O.MSG)

Format: HEADER.ID|STATUS|FT.ID|OFS.RECORD|PAID.AMOUNT|OVERRIDE|

Status Code	Meaning
0000	Success (FT posted)
0006	Generic reject — insufficient balance OR channel restriction OR OFS posting failure
0011	Insufficient balance — Business Banking / BB.MCP overdraft-specific variant
3003	Posting restricted — customer industry / posting-restriction match

Gap risk: these 4 status codes are CIB-custom, not standard T24/IRIS error codes. A full error code mapping table is required against whatever IRIS returns natively.

5. Config Tables This Routine Depends On (must be migrated/replicated)

Table	Purpose
CIB.API.OVERRIDE	Per-channel: auth-flag behavior, OD-flag behavior, extra validation routine name, override-check mode, restricted posting codes & descriptions
ST.CIB.FI.FT.PARAM	Per-channel: list of customer industry codes subject to Business Banking restrictions
CIB.API.FT	Idempotency/audit store — one record per MSG.ID with status history
CIB.ONLINE.FTS	SMS/notification trigger log

6. External Routines Called (separate services to audit individually)

Routine	Purpose
CIB.API.VALIDATE.FT	Channel/txn-type restriction check (API channel)
GET.NET.AVL.BL	Balance computation
CIB.MT.MX	Name transliteration for SWIFT MT → MX
IBANService.ValidateIBAN	Standard T24 IBAN validation
EXCHRATE	Cross-currency conversion
CIB.MSG.AUTHORIZE.SC	Auto-authorization
CIB.CHECK.SMS.RUN	SMS trigger check
Dynamic routine (CIB.API.OVERRIDE<API.OVR.R TN.NAME.OT>)	Channel-specific extra validation, resolved at runtime — needs discovery of which channels populate it

7. Recommended TPH/IRIS Migration Checklist for This Service

- ☐ Confirm IRIS "Create Outward Transfer" API request schema and map every one of the 51 input fields above

- ☐ Reproduce idempotency behavior (cached-success replay) or confirm IRIS's native idempotency key handles it equivalently
- ☐ Reproduce/relocate the CIB.API.VALIDATE.FT channel-restriction check
- ☐ Reproduce Business Banking industry + posting-restriction rule engine (config-driven)
- ☐ Reproduce available-balance + overdraft (OD.FLAG/AUTH.FLAG) logic, including cross-currency conversion
- ☐ Reproduce BB.MCP dry-run overdraft charge check
- ☐ Reproduce commission/charges default logic (OUR/BEN/SHA + account-length override)
- ☐ Reproduce IBAN validation and MT → MX name transliteration
- ☐ Reproduce auto-authorization logic and confirm per-channel config still applies
- ☐ Build full error code mapping table: 0000/0006/0011/3003 (CIB-custom) → IRIS native codes
- ☐ Confirm special-character sanitization on input is no longer needed (should be handled naturally by JSON in IRIS) but validate no silent data loss for those characters in narratives/names
- ☐ Discover and separately audit all channel-specific dynamic extra-validation routines referenced via CIB.API.OVERRIDE

8. Channels Confirmed From Code

API, VOS, BPM, BB.MCP, plus any channel with a row in CIB.API.OVERRIDE (config-driven, not hardcoded in this routine) — recommend pulling the full CIB.API.OVERRIDE table contents to get the definitive channel list.

9. Code Evolution / Technical Debt Notes

The routine carries visible attribution comments spanning multiple years and authors, indicating incremental patching rather than periodic redesign. This is a signal that some logic may be undocumented tribal knowledge rather than formally specified requirements — each patch below should be traced back to its original change request where possible before being reproduced on TPH.

Marker / Author Tag	Approx. Date	What It Touched (from code)
MCBCS	14-Jun-2018	R18 upgrade retrofit — large-scale replacement of legacy @-notation and '=' comparisons with EQ/NE/GT/LT and F.READ calls; introduced FUNDS.TRANSFER\$NAU handling
NGDE	2017–2018 (multiple)	Added CHARGES.ACCT.NO, CREDIT.THEIR.REF, DEBIT.VALUE.DATE, VERSION.NAME parameters; BPM-specific charge/version logic
CIB20200607	Jun-2020	VOS channel handling — UETR generation, version name override, SRC.CHANNEL truncation to 4 chars
RANIA	18-Sep-2022	Introduced BB.FLAG / Business Banking industry-restriction logic, ST.CIB.FI.FT.PARAM lookup, posting-restriction (3003) rejection path
MENNA	Jun-2023 / Jan-2024	Added OLD.CBE.CODE, NEW.CBE.CODE, SELL.CODE, CBE.NAR.AR fields — likely a Central Bank of Egypt reporting requirement
HEND	15-Aug-2024 / 25-Dec-2024	BB.MCP overdraft charge dry-run check (CHECK.BB.MCP.OVERDRAFT.CHARGE); beneficiary-name reformatting logic for MT → MX and multi-part names

Recommendation: treat each dated block above as a separately traceable requirement when writing TPH/IRIS acceptance criteria, rather than migrating "the routine" as one undifferentiated unit.

10. Open Questions / Assumptions for the Team

- What are the exact rules inside CIB.API.VALIDATE.FT (channel/txn-type restriction check)? Not visible in this routine — needs its own source review.
- What industry codes and posting-restriction codes are currently configured in ST.CIB.FI.FT.PARAM and CIB.API.OVERRIDE? Need a full data export, not just the logic.
- Is CBE reporting (OLD.CBE.CODE / NEW.CBE.CODE / SELL.CODE / CBE.NAR.AR) still required in the same form on TPH, or does IRIS have native CBE reporting fields?
- Does IRIS provide native idempotency-key handling equivalent to the CIB.API.FT cached-replay behavior, or must this be built in the orchestration layer?
- Who owns confirmation that auto-authorization (CIB.MSG.AUTHORIZE.SC) is an approved control exception, and does Compliance need to re-approve it for TPH?
- Is the MT ↔ MX beneficiary-name transformation (CIB.MT.MX) still needed, or does IRIS handle ISO 20022 name formatting natively?
- Which channels currently populate the dynamic extra-validation routine (CIB.API.OVERRIDE<API.OVR.RTN.NAME.OT>), and what does each one do?

11. Review & Sign-off

Role	Name	Date	Comments
Prepared by			
T24 Technical Reviewer			
TPH/IRIS Solution Architect			
Business/Product Owner			
Compliance/Controls (for auto-auth & OD logic)			